CREMA

Zero-Error Implementation Plan

Repository audit • phased remediation • verification gates • submission evidence

Target repository: https://github.com/parth-apsit/Crema

Executive Summary

Crema is a Next.js 16 / React / TypeScript application using Firebase, Docker Compose, GitHub Actions, Prometheus and Grafana. The repository already contains most structural components required by the assignment. The highest-value approach is to fix implementation/documentation mismatches, prove the stack from a clean checkout, secure the operational configuration, and make deployment claims match what actually happens.

Audit area	Finding	Next step
Application/source	Application structure and routes are present.	Run fresh install, lint, typecheck, tests, production build and representative user-flow checks.
CI	Recorded CI run passed lint, tests and Next build at commit ba8b7e82.	Add typecheck and container smoke test; capture new run against final commit.
Deployment	deploy.yml builds/pushes GHCR image; VM rollout commands are a commented placeholder.	Implement genuine SSH deployment, or explicitly scope the deliverable to GHCR publication.
Docker/Compose	Multi-stage Dockerfile, app healthcheck, web + Prometheus + Grafana already configured.	Validate clean build; use consistent environment variables; limit exposed ports.
Monitoring	Prometheus and Grafana provisioning is present.	Validate scrape target/dashboard with generated traffic; improve histogram and error-rate metrics.
Security	Firestore rules exist; main branch is unprotected; Grafana default credentials are hard-coded.	Test rules, protect main, remove defaults and scope operational network access.
Documentation	Evidence and runbooks exist but contain mismatched/stale commands and credentials.	Update runbook, architecture, evidence list, screenshots and report with final actual outputs.

What “zero-error” means in this plan: no known failing checks or contradictory instructions, repeatable build/run procedures, tested expected failures, and evidence tied to the final commit. No implementation plan can guarantee zero defects in all environments; each gate below must actually pass before it is marked complete.

1. Repository Baseline and Verified Facts

- Default branch: main. Latest inspected commit: ba8b7e8278d69800c961e2f618e4e65992088e64 (merge commit dated 15 October 2026).
- GitHub Actions CI run 38790418198 is recorded as successful. Its steps show npm install, ESLint, Vitest, and Next.js build completed successfully.

- The deployment workflow run 38790418211 is recorded as successful, but source configuration shows the VM deploy stage only prints setup instructions. A green job therefore does not establish a VM rollout.
- The repository includes Dockerfile, docker-compose.yml, Prometheus configurations, Grafana provisioning/dashboard, health and metrics API routes, Firestore rules, testing guide, run guide and submission evidence report.
- Current package scripts: dev, build, start, lint = eslint, test = vitest run. Runbook references to test:watch or test:coverage should not be presented as working until those scripts are added.
- Current default branch is reported unprotected by GitHub. The latest checked in workflow names/steps are the source of truth rather than narrative descriptions alone.

2. Priority and Change-Control Rules

Priority	Meaning	Order
P0 — block release	Security, misleading deployment evidence, or production release controls.	Complete before final demo/release.
P1 — correctness	Metrics correctness, test coverage, readiness, docs and runbook agreement.	Complete before final evidence capture.
P2 — polish	Expanded dashboard panels, coverage polish, accessibility and non-essential workflow optimizations.	Complete if time remains; do not destabilize a validated system.

- Create a feature branch before changing files. Make one logically scoped change per commit and avoid uploading credentials, .env.local files, service account JSON, private SSH keys or tokens.
- After each phase, run its gate before proceeding. Do not mark a checklist item complete based on configuration presence alone.
- Keep a known-good commit and image digest. Avoid broad dependency upgrades during remediation unless a specific compatibility issue requires one.

3. Phased Implementation Roadmap

Phase 0 — Establish a reproducible baseline

- Record the starting commit: ba8b7e8278d69800c961e2f618e4e65992088e64, branch state and current working-tree status.
- Review and back up local environment configuration without copying secrets into evidence/report files.
- Run the existing workflow locally from a clean checkout using the Node version specified by CI (Node 22).
- Save results and distinguish pre-existing warnings from failures. Do not suppress warnings globally just to make output appear clean.

```
git status --short
git rev-parse HEAD
node --version
npm --version
npm ci --legacy-peer-deps
npm run lint
npx tsc --noEmit
npm test
npm run build
```

Phase 1 — Correct scripts, CI and branch discipline

- Keep package.json as the canonical command list. README currently describes lint as next lint, while the real command is eslint.
- Add a package script: "typecheck": "tsc --noEmit". Add test:watch or test:coverage only if wanted and configure the necessary tooling; otherwise remove those references from documentation.
- Add npm run typecheck to .github/workflows/ci.yml. Maintain Node 22 alignment across CI, Docker and local instructions.
- Ensure the lockfile and npm configuration are committed. Preserve npm ci reproducibility and avoid mixing npm install generated changes without review.
- Protect main: require pull request review and successful CI checks; block force-pushes. Test with a draft pull request and confirm the check appears.
- Use explicit least-privilege GitHub Actions permissions and concurrency for production deployments.

```
npm run lint
npm run typecheck
npm test
npm run build
```

Phase 2 — Configuration and secret hygiene

- Keep .env.example as an empty template. Create .env.local locally and ensure .env*, real secrets and service credentials remain excluded by .gitignore and .dockerignore.
- Review Firebase NEXT_PUBLIC_* variables: they are intended to be browser-visible config, not secret storage. Remove automatic defaults that can silently point to an unintended project.
- Validate required build-time public Firebase settings. Fail with an actionable message when required settings are missing, rather than producing an image that starts but cannot authenticate/connect correctly.
- Use a unique Grafana admin password supplied by environment/secret configuration. Remove admin/admin and crema2026 hard-coded credentials from examples; rotate credentials if a real deployment exposed them.
- Avoid printing secret values in workflows, shell traces, application logs or screenshots. Keep private server-only values separate from NEXT_PUBLIC variables.
- Bind Grafana/Prometheus to loopback for local usage, or expose through secured access controls. Open only the required application ingress ports on the VM firewall.

Phase 3 — Docker build and runtime correctness

- Preserve the three-stage build (deps, builder, runner), Node 22 Alpine base and Next.js output: standalone setting.
- Ensure .next/standalone/server.js and .next/static are present in the final image; ensure public assets required by the app are copied.
- Keep the healthcheck pointed at http://127.0.0.1:3000/api/health in both Dockerfile and Compose. Check the service binds to 0.0.0.0:3000.
- Use docker compose config --quiet before starting. Run a clean no-cache build once after changes; use cached builds after the baseline is established.
- Run as non-root and verify no unnecessary build files or env files are copied into the runtime image.
- Add a CI container smoke test: build image, launch it with test configuration, poll health endpoint with a deadline, inspect logs, then stop/remove the container even on failure.

```
docker compose config --quiet
docker compose build --no-cache
docker compose up -d
```

```
docker compose ps
docker compose logs --tail=100 crema-web
curl -fsS http://localhost:3000/api/health
curl -fsS http://localhost:3000/api/metrics
```

Phase 4 — Make Prometheus and Grafana metrics meaningful

- Retain Prometheus target crema-web:3000 and metrics path /api/metrics within the Compose network; keep localhost:3000 only for a Prometheus instance running on the host.
- Add a Prometheus histogram for request duration with finite buckets and valid bucket/sum/count series. The current summary has only sum/count, so quantiles cannot be calculated from the emitted data.
- Ensure instrumentation covers the actual HTTP routes you intend to measure. Document current in-memory aggregation and reset-on-restart behavior; it is not multi-replica aggregation.
- Use bounded labels. Do not put user IDs, order IDs, query strings or arbitrary dynamic path values into metric labels.
- Clarify whether health checks and metrics scrapes count as application traffic; exclude them from customer-facing traffic panels or label them clearly.
- Provision panels for request rate, 5xx rate, p95 latency (after histogram implementation), uptime, and heap memory. Add threshold/units and useful legends.
- Verify Prometheus /api/v1/targets reports the app target UP with an empty lastError. Generate normal test traffic and a controlled test error; verify panel/query values change.
- Persist Prometheus/Grafana data only when needed and document the chosen retention policy.

```
# Requests/sec
sum(rate(http_requests_total[5m]))

# 5xx rate ratio (after 5xx instrumentation is present)
sum(rate(http_requests_total{status=~"5.."}[5m]))
/ clamp_min(sum(rate(http_requests_total[5m])), 1e-9)

# P95 latency (only after histogram bucket instrumentation exists)
histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))
```

Phase 5 — Health, readiness and application/security tests

- Define /api/health as liveness: confirms the process can answer. Add a separate readiness/dependency check if Firebase availability must gate traffic; avoid costly database reads on every liveness probe.
- Add focused integration/security tests for logged-out access, customer/admin separation, customer order ownership, protected inventory changes, coupon conditions and concurrent inventory deduction.
- The Firestore rules restrict many operations, but order creation primarily verifies the submitted userId. Validate sensitive amount/status/quantity fields and ensure clients cannot choose arbitrary prices or grant themselves business privileges.
- Run Firebase Emulator rules tests for allowed and denied operations before treating the rules as hardened.
- Run one customer checkout and one admin order-fulfillment flow against a test project, including refresh/reconnect behavior and visible failure handling.

Phase 6 — Correct deployment pipeline

- Choose the submission claim before implementation: (A) image delivery to GHCR, or (B) actual deployment to a VM. Do not claim B until a VM rollout is verified.
- Build and push the image only after CI gates pass. Tag by commit SHA, record digest and use that immutable artifact during rollout.

- Enable SSH rollout using masked repository secrets after confirming host/user/key/path, Linux Docker access, target firewall and a deployment directory controlled by the deployment account.
- On the target, log in to GHCR as necessary, pull the exact digest, update only the intended Compose service, wait for health and check recent logs.
- Add a timeout and failure exit for health validation. Preserve previous image digest and implement documented rollback; do not leave a failed new deployment running silently.
- Use workflow concurrency to avoid overlapping releases and add a minimal post-deploy HTTP check from the appropriate network location.
- If VM deployment cannot be completed, report successful CI + GHCR publication as container delivery and explicitly list VM auto-deployment as not implemented.

Phase 7 — Documentation, report and screenshots

- Update README: correct the lint description and credentials, list accurate environment setup, supported Node version and all working commands.
- Update docs/how-to-run.md: remove stale/insecure instructions, distinguish Windows/WSL2 from Linux commands, document current scripts, host/port bindings and secret setup.
- Update docs/implementation.md: reconcile Node 20 versus Node 22, correct repo identity, remove claims unsupported by the current pipeline, and fill in real member contributions.
- Update docs/submission_evidence.md only after a fresh verification. Label historical logs as historical and replace placeholders with current evidence.
- Update architecture documentation if the real deployment topology differs from the diagram.
- Never stage .env.local, SSH keys, access tokens, real credentials, unredacted env output or personal account details in screenshots.

Phase 8 — Final release gate

- Pull the final branch into a clean directory or fresh environment and repeat all local checks.
- Confirm latest GitHub Actions CI is green for the final commit. If performing VM deployment, confirm it is deployed by SHA/digest and is healthy from the expected client path.
- Verify Prometheus target and query results, Grafana datasource and all required panels after the final stack starts.
- Capture application, Git history, CI, test, Docker, deployed/accessible application, Prometheus and Grafana evidence.
- Rehearse the demonstration from a clean start and retain a fallback command path in the runbook.

4. Exact Final Verification Commands

4.1 Source validation

```
git status --short
git rev-parse HEAD
node --version
npm ci --legacy-peer-deps
npm run lint
npm run typecheck
npm test
npm run build
```

4.2 Container stack

```
docker compose config --quiet
docker compose build --no-cache
docker compose up -d
docker compose ps
docker inspect crema-web --format "{{json .State.Health}}"
curl -fsS http://localhost:3000/api/health
curl -fsS http://localhost:3000/api/metrics
curl -fsS http://localhost:9090/api/v1/targets
docker compose logs --tail=100
```

4.3 Observability acceptance queries

```
curl -fsS "http://localhost:9090/api/v1/query?query=up"
curl -fsS "http://localhost:9090/api/v1/query?query=http_requests_total"
curl -fsS -u "$GRAFANA_USER:$GRAFANA_PASSWORD" http://localhost:3001/api/health
```

Run curl.exe instead of curl where PowerShell command alias behavior interferes. Quote URLs and credentials appropriately for the shell in use. Do not echo secrets as part of troubleshooting.

5. Evidence Matrix for Submission

ID	Screenshot / artifact	What it proves	Capture timing
E01	GitHub repository root + commit history	Source repository and meaningful commit history exist.	Final code freeze
E02	package.json and CI workflow	Scripts and automated validation configuration.	After CI edits
E03	GitHub Actions CI summary	Lint, typecheck, tests and build passed.	Final commit
E04	Vitest output	Test suites and test count passed.	Final commit
E05	Dockerfile + Compose config	Multi-stage image and multi-service topology.	After final config
E06	docker compose ps and health inspect	All required containers running; app is healthy.	Fresh stack start
E07	Browser app + /api/health	App is accessible and endpoint returns valid JSON.	Fresh stack start
E08	/api/metrics response	Prometheus exposition format and real metrics.	After sample traffic
E09	Prometheus Targets / query	App target is UP and queries have data.	After scrape delay
E10	Grafana full dashboard	Provisioned data source and meaningful populated panels.	After sample traffic
E11	GHCR package/tag/digest	Built image was published.	After publish run
E12	VM service status + external health	Actual deployment and accessibility (only if VM deploy implemented).	After deployment
E13	Application test-flow screenshots	Customer/admin core flows work.	Final smoke test
E14	Final presentation/demo outline	End-to-end pipeline can be explained and demonstrated.	Before submission

6. Final Acceptance Checklist

☐ Fresh dependency install succeeds from the lockfile.

☐ ESLint, TypeScript check, all tests and production build exit with code 0.

☐ GitHub Actions CI is green on the final commit and required branch protection is active.

☐ Docker image builds cleanly and runs as the intended non-root user.

☐ Compose starts all three services; application is healthy and reachable.

☐ Prometheus application target is UP; metrics update under generated traffic.

☐ Grafana login is not using a repository-shipped default password; monitoring ports are appropriately scoped.

☐ Histogram-based latency and error-rate panels are backed by real exported series, or explicitly marked as outside the current verified scope.

☐ Firestore allow/deny cases are automated and sensitive order fields cannot be trusted directly from the client.

☐ Deployment claims match reality: GHCR-only delivery or verified VM rollout, never a placeholder represented as deployment.

☐ README, runbook, implementation report, evidence report and screenshots agree with final code and current commit.

☐ No credentials or sensitive environment files are tracked or included in evidence.

☐ Live demonstration is rehearsed with a documented recovery/fallback route.

7. Recommended order of execution

1. Create a branch and record the clean baseline.
2. Fix script/runbook inconsistencies and add type-check CI.
3. Remove unsafe defaults, scope monitoring ports and protect main.
4. Build and smoke-test the Docker image from CI and local Compose.
5. Improve metrics instrumentation and validate the dashboard.
6. Add focused Firebase rules and application security tests.
7. Implement and test actual VM rollout, or explicitly settle for GHCR delivery.
8. Update reports; capture final screenshots and commit evidence.
9. Re-run the complete release gate and rehearse the live presentation.

Sources inspected

- Repository: https://github.com/parth-apsit/Crema
- CI workflow: https://github.com/parth-apsit/Crema/blob/main/.github/workflows/ci.yml
- Deploy workflow: https://github.com/parth-apsit/Crema/blob/main/.github/workflows/deploy.yml
- Dockerfile: https://github.com/parth-apsit/Crema/blob/main/Dockerfile
- Compose: https://github.com/parth-apsit/Crema/blob/main/docker-compose.yml
- Monitoring: https://github.com/parth-apsit/Crema/tree/main/prometheus and https://github.com/parth-apsit/Crema/tree/main/grafana/provisioning
- Current recorded CI run: https://github.com/parth-apsit/Crema/actions/runs/38790418198
- Existing submission evidence: https://github.com/parth-apsit/Crema/blob/main/docs/submission_evidence.md
- Existing process log: https://github.com/parth-apsit/Crema/blob/main/docs/devops-process-log(15%20Oct).md